

vue2不维护了，新项目或者改版重构的项目继续使用vue2会有什么问题？

vue2不维护了？

Vue 2 已经到达终止支持 (EOL) 时间

Vue 2.0 发布于 8 年前的 2016 年。这是 Vue 成为主流框架过程中的一个重要里程碑。目前的许多 Vue 用户都是在 Vue 2 时代开始使用 Vue 的，并用它构建了许多出色的项目。

然而，同时维护两个大版本对我们来说是无法长期持续的。随着 Vue 3 及其生态系统的逐步成熟，团队是时候继续前进，将精力集中在最新的大版本上了。

Vue 2 已于 2023 年 12 月 31 日达到终止支持时间。它不再会有新增功能、更新或问题修复。不过，它依然可以从所有现有的分发渠道 (CDN、包管理器、GitHub 等) 上获得。

如果你要启动一个新项目，请从 Vue 的最新版本 (3.x) 开始。我们也强烈建议当前的 Vue 2 用户了解(迁移指南)，尽管我们理解并非所有用户都有足够的资源或动力进行升级。如果你需要继续使用 Vue 2，但又对不在维护的软件有合规性或安全性要求，请移步了解 HeroDevs 对 Vue 2 的无限期支持 (NES)。

下一步计划

自 2022 年 2 月 7 日起，Vue 3 已成为 Vue 的默认版本。

已经迁移的用户可以享受到：

更小的包大小和更快的渲染速度带来的更好的性能。

增强的 TypeScript 支持，使大规模应用开发更轻松。

基于 Proxy 的更高效的响应性系统。

新的内置组件，如 Fragment、Teleport 和 Suspense。

改进的构建工具支持和 Vue Devtools 体验。

.....以及更多！

如果条件允许，请考虑迁移！

参考 vue2 官方文档：<https://v2.cn.vuejs.org/eol/>

新项目或者改版重构的项目继续使用vue2会有什么问题？

Vue2长期以来本身存在的一些问题

1. 响应式系统限制：

- Vue 2 使用 `Object.defineProperty()` 来实现数据响应式绑定。这意味着只有对象属性被声明为响应式的，当访问未初始化的属性时，Vue 不能检测到它们的添加或删除。例如，若先创建了一个空对象，然后给这个对象动态添加新属性，新添加的属性不会触发视图更新。
- 对于数组，Vue 2 提供了变异方法（如 `push`、`pop`、`shift`、`unshift`、`splice`、`sort` 和 `reverse`）和 `$set`、`$delete` 方法来确保响应式，但直接修改数组索引或使用非变异方法（如 `arr[index] = newValue`）不会触发视图更新。

2. 深度观测性能损耗：

- Vue 2 默认只对对象第一层属性进行响应式处理。若需要深层次的对象属性也响应式，必须显式地设置 `deep: true`，但这会导致性能开销增大。

3. Mixins 混入机制：

- Vue 2 中 mixins 混入机制虽有效实现了组件业务逻辑的复用，但也带来了命名冲突、依赖管理复杂、可读性与可追踪性下降、过度耦合、生命周期管理挑战等潜在问题。

4. 模板语法限制：

- Vue 2 模板语法虽然直观易用，但在某些复杂逻辑场景下可能不够灵活，需要借助计算属性或辅助函数来完成。

5. API设计与扩展性：

- Vue 2 的 Options API 设计在大型应用中有时显得冗余和不易组织，特别是在状态管理和组件复用方面，Composition API（最初并未包含在Vue 2中，后来通过 `@vue/composition-api` 插件提供，而在Vue 3成为核心特性）的引入旨在解决这个问题。

6. TypeScript集成：

- Vue 2 对 TypeScript 的原生支持不如 Vue 3 完善，尽管可以通过社区的努力得到改善，但Vue 3 从设计之初就考虑了对 TypeScript 的更好兼容。

7. 性能优化：

- Vue 2 在编译优化和运行时效率上仍有提升空间，Vue 3 通过改进编译器和引入Proxy等方式显著提升了性能和内存占用。

Vue 3 在很大程度上解决了上述问题，特别是通过引入基于ES6 Proxy的响应式系统、Composition API 等重大改进，使得Vue在现代JavaScript开发中更具竞争力和灵活性。不过，在Vue 2停止维护前，许多问题都有相应的社区解决方案或最佳实践以缓解这些问题的影响。

Vue 2 在正式停止维护后，对于新项目或改版重构的项目选择继续使用 Vue 2 可能会面临以下几个潜在问题和挑战：

1. 安全更新：

- 官方不再发布针对Vue 2的安全补丁，如果未来出现新的安全漏洞，则项目存在安全风险。
- 随着时间的推移，Vue2可能面临越来越多的安全风险。新的安全漏洞和攻击方式可能不会被及时修复，从而增加项目遭受攻击的风险。

2. 浏览器兼容性：

- 随着浏览器的发展，Vue 2 可能无法及时适应新的浏览器特性或变化，导致在新版浏览器中出现兼容性问题。

3. 生态系统（社区）支持萎缩：

- Vue 生态中的第三方库和插件开发者可能逐渐转向对 Vue 3 的支持，Vue 2 的一些库和插件在未来可能不再维护或更新，造成依赖项过时或缺失 bug 修复。
- 随着Vue3的推出和普及，Vue2的社区支持可能会逐渐减少。这意味着在寻求帮助、解决问题或学习新技巧时，可能会发现相关的资源和讨论不如Vue3丰富。

4. 技术落后：

- Vue 3 引入了许多新特性和优化，比如Composition API、更好的Tree-Shaking能力、更快的编译速度和运行时性能提升等。**坚持使用 Vue 2 就意味着错过这些技术和性能优势。**
- 随着前端技术的不断发展，新的标准和工具不断涌现。Vue2可能无法与最新的浏览器特性、性能优化工具或构建工具完全兼容，这可能会限制项目在未来的技术升级和扩展能力。

5. 长期维护成本：

- 如果项目生命周期较长，随着技术栈的老化，团队成员的学习资源和技术支持会减少，可能导致维护成本增加。

6. 团队技能发展和招聘：

- 业界趋势和技术发展方向倾向于Vue 3，团队掌握新技术的能力和市场竞争力可能会受到影响。
- 随着Vue3的普及，越来越多的开发者开始学习和使用Vue3。如果继续使用Vue2，可能会在招聘新成员或与其他团队协作时遇到困难，因为具备Vue2经验的开发者可能会相对较少。

因此，虽然现有Vue 2项目在一段时间内仍可正常运行，但对于新建或重构的项目，**为了获得更好的长期支持和更先进的技术方案，推荐采用Vue 3进行开发。**当然，在决定是否迁移时，也需要考虑项目实际情况、预算、计划时间表以及团队的技术熟练度等因素。